

用 LLM + Obsidian 构建个人知识库

基于 Karpathy 的 LLM Knowledge Bases 工作流

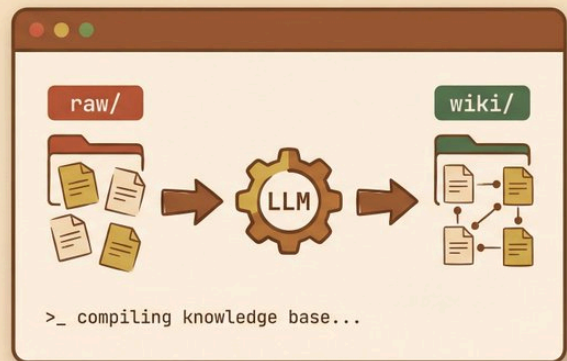
Karpathy

Obsidian

LLM

知识工程

编译



Karpathy 今天发了一条推文，讲他怎么用 LLM 构建和管理个人知识库。

我反复读了几遍。不是因为什么新概念，是因为他做的事和我这几个月在 Obsidian 里瞎折腾的东西，结构上撞了。但他用了一个词来概括，我之前一直没找到：

编译 (Compile)。

把原始资料“编译”成结构化知识。Obsidian Vault 是代码仓库，LLM 是编译器。

这个类比一出来，好多之前说不清的感觉突然有名字了。

你的笔记库在腐烂

我之前写过关于 Obsidian + AI 的文章：

评论区里有一类反馈反复出现：

我也在用 Obsidian，攒了几百篇笔记，但越攒越乱。我知道里面有好东西，但找不到。

我太理解这种感觉了，因为我自己就是这样。

剪裁了一堆文章，记了不少想法，攒了很多链接。三个月后回头看，大部分变成了死文件。你记不住在哪，也记不住讲了什么。传统解法是“定期整理”，加标签、补链接、重新分类。我试了两周就放弃了。靠人来维护笔记库，就跟靠人来回归测试一样，理论上可以，实际上不可能持续。

Karpathy 换了个思路：别让人整理，让 LLM 整理。而且不是随便整理，是用软件工程的方式来搞。有输入层，有编译层，有输出层。有索引，有健康检查。

把知识库当代码仓库管

如果你写过代码，Karpathy 这套东西你会秒懂。

软件工程	→ 知识库工程
src/	→ raw/ (原始资料)
build/	→ wiki/ (知识条目)
logs/	→ outputs/ (问答归档)
编译器	→ LLM
IDE	→ Obsidian
Lint / CI	→ 健康检查
增量编译	→ 只处理新增/变更的 raw

我是开发出身，第一眼看到这张表的反应是：这不就是 CI/CD 的知识库版本吗？

核心就三件事。

分层。原始资料、编译产物、运行时输出，三层分开。你不会把 .class 文件和 .java 文件混在一起，笔记也一样。

增量。不用每次全量重建。每天新进来几篇文章，增量编译就好。

可追溯。每个知识条目能追到原始来源，每个摘要保留原文引用。出了问题能查到根源。不是“我好像在哪里看过这个说法”，是“来源在 raw/articles/2026-03/ 那篇里，第三段”。

我的落地方案：三层目录 + Claude Code 当编译器

说完类比说实操。我把 Karpathy 的方法论落到了自己的 Vault 里，目录结构长这样：

```
Vault/
├─ raw/                                # 原始资料，不改
│   ├─ articles/                       # Web Clipper 剪藏
│   ├─ podcasts/                       # Podwise 导出
│   └─ papers/                         # 论文
│
├─ wiki/                              # 编译产物，LLM 维护
│   ├─ indexes/                       # 索引（来源清单、概念清单、术语表）
│   ├─ concepts/                     # 概念条目
│   └─ summaries/                   # 逐篇摘要
│
├─ outputs/                           # 运行时输出
│   ├─ qa/                           # 问答沉淀
│   └─ health/                       # 健康检查报告
│
├─ x/                                 # X 平台成品
├─ 公众号/                           # 公众号成品
└─ 小红书/                           # 小红书成品
```

我保留了平台内容目录，因为我的知识库不只是研究用的，还要出内容。成品是给读者看的，wiki 是给自己看的，这两个不能混。

摄取：三个入口

1. Web Clipper 管网页文章。一键保存，模板里强制写入 source_url、author、published、tags。这步很重要，没有元数据的剪藏跟没剪一样。
2. Podwise 管播客。自动转录加 AI 摘要，导出 Markdown。我之前专门写过一篇讲这个：
3. 手动剪藏 管 X 推文和零散内容。Claude Code 有个 Skill 叫 baoyu-url-to-markdown，丢个链接进去，几秒变干净的 Markdown。

编译：这步最关键

攒了 5 到 10 篇 raw 之后，就可以做第一次“编译”了。在 Obsidian 里打开 Claudian，跟 Claude 说：“读一下 raw/articles/ 里最近新增的文章，为每篇生成摘要，提取概念，更新索引。”然后让它干就行了。

具体来说干三件事：

逐篇摘要。每篇 raw 文档产出一份结构化摘要，包括核心结论、关键证据、疑点、术语。存到 wiki/summaries/。

概念抽取。从摘要里提概念，映射到 wiki/concepts/。新概念就建条目，老概念就补新证据。

索引更新。自动维护 [All-Sources.md](#) 和 [All-Concepts.md](#)。

质量怎么保证？靠 [CLAUDE.md](#)。我在里面写了编译规范：摘要的结构模板、概念条目该有哪些字段、命名规则。Claude 每次启动都读，输出就稳了。

第一次编译可能要花半小时到一小时。但跑通一次之后，后面增量编译很快，因为只处理新增的 raw。

让每次对话都变成库存

Karpathy 方法论里有一个设计我觉得特别妙：Output 落文件。

以前我用 AI 的方式是：问一个问题，得到答案，关掉。下次有类似问题再问一遍。答案全在聊天记录里，等于没有。

现在改了。每次对知识库做复杂提问，结果以 Markdown 文件存到 outputs/qa/：

```
---
question: "RAG 和轻量索引的适用边界？"
asked_at: 2026-04-03
sources:
  - [[S-001 MotherDuck Obsidian RAG]]
  - [[C-042 RAG]]
---
```

RAG vs 轻量索引

TL;DR

规模在万级 note 以下，轻量索引够了。
语义搜索需求强或规模更大，上 RAG。

结论

...

证据

(链接回原始来源)

不确定性

...

三个月下来我积累了几十份这样的 Q&A 文件。它们本身就是知识条目，因为带推理过程和原始来源。下次遇到类似问题，Claude 直接读已有的 Q&A，不用重新推导。

你每跟 AI 聊一次，知识库就增加一层。

健康检查：给知识库做体检

这步是大部分人不会想到的。

你的笔记库里有没有这些情况：同一个概念在三个地方定义不一样；某个条目只有标题没有正文；一堆笔记孤零零没有任何链接指向。

我之前也没在意。直到有一次搜到一条笔记，写的结论跟另一条完全矛盾，搞不清哪个是对的。这才意识到知识库也有“技术债”，不处理的话时间越长越乱。

Karpathy 的建议是定期做 Health Checks。我把它做成了每周任务，让 Claude 检查三样东西：

1. 一致性。wiki/concepts/ 里有没有定义冲突？比如"RAG"在一个地方叫"检索增强生成"，另一个地方变成了"向量数据库搜索"。
2. 完整性。哪些概念条目缺定义、缺例子、缺来源？
3. 孤岛。哪些笔记入链出链都少于 2？该连到哪？

报告存到 outputs/health/，每周一份。做了几周之后最大的感受是：搜到一条笔记，敢直接用了。以前还得想想：“这玩意靠谱吗”。

别上来就搞 RAG

这点必须单独说，因为太多人在这儿可能会走弯路。

一提"AI + 笔记"，很多人的第一反应是搭 RAG：选 Embedding 模型、搭向量数据库、调切片策略。整套架构搞了一个月，笔记库里还是只有 20 篇文章。

Karpathy 推荐这样做：知识库规模不大的时候（100 篇文章，40 万词左右），维护几个索引文件就够了。LLM 先读索引定位，再直接阅读相关内容。简单、可靠、零额外成本。

等你的笔记真的过了一万条，搜东西开始找不到、找不全了，再考虑 RAG。MotherDuck 有篇博文讲了用 DuckDB 给 Obsidian 做向量检索的完整方案，到时候再看不迟。

先跑通流程，再优化基础设施。这道理写代码的人应该都懂，但轮到自己搭知识库的时候就忘了。

两周跑通最小闭环

如果你想试，不需要什么额外工具。Obsidian + 任何你用的 LLM 就行。

第一周：搭 raw → wiki 的最小循环。建三个目录（raw / wiki / outputs），装 Web Clipper 配好模板，开始剪裁。攒够 5 到 10 篇后做第一次编译：生成摘要、提概念、建索引。

第二周：让 Q&A 开始积累，启动第一次健康检查。所有复杂问答的结果存到 outputs/qa/。让 LLM 扫一遍 wiki/，输出第一份健康报告。按报告补缺失、加链接。

两周之后你有一个能持续运转的小系统。规模不重要，流程跑通了就行。后面就是往 raw/ 里不断喂素材，让编译器持续工作。

知识库的"GitHub 时刻"

Karpathy 推文最后一句话是：

这套东西目前仍然像一堆 hacky scripts，但有空间做成 incredible new product。

我想到了 2006 年前的版本控制。那时候也是 svn、cvs、git 命令行，只有程序员在用。然后有人把它做成了 GitHub，整个协作方式都变了。

个人知识库可能正在类似的节点。今天它是 Obsidian + LLM + 手搓脚本的组合，看起来还很粗糙。但底层范式已经有了：把知识当代码来管理。有输入，有编译，有产物，有测试。

如果你是程序员，好消息是你不需要学任何新东西。代码仓库怎么管，知识库就怎么管。你积累了这么多年的工程直觉，终于可以用在自己的笔记上了。

如果你不写代码，也别被“编译”这个词吓住。说白了就是：让 AI 帮你定期整理笔记，而且比你自己整理得好得多。因为它不累，不忘，不偷懒。

别让你的笔记腐烂。让它们被编译。

延伸阅读（我的 Obsidian + AI 系列）：

- [《如何基于 Obsidian 和 Claude 打造 AI 时代超级大脑？》](#) → 把笔记库变成内容工厂
- [《Obsidian + Claude Code：你的笔记库就是你最强大的 AI 上下文》](#) → 笔记库作为 AI 上下文的价值
- [《我用 Podwise 把播客变成了可检索的知识库》](#) → 播客信息摄取的完整流程

你现在是否也在搭自己的知识库系统？

欢迎关注 --> @yanhua1010，评论区聊聊你的方案。这个领域现在很早期，说不定你的实践就是未来某个产品的原型。

原文链接：[查看原文](#) 抓取时间：2026-07-28T00:29:45.984804+08:00